

web

Wednesday, 1 April 2026 16:58



Anthropic通过一个57MB的sourcemap文件，泄露了Claude Code CLI完整的TypeScript源码。

这是一次典型的配置失误：打包时忘了排除.map文件，npm把原始源码直接发了出去。

不过，对正在创业做Agent的开发者来说，这次泄露是一次免费的架构课。

本文所有信息均来自开发者们对源码的公开分析。

1. Claude Code是一套有11层架构的系统

Claude Code的架构，被设计者描述为"极度防御的分层系统" (extremely defensive layer system)，是一个持续运转的循环：

用户消息 → 上下文组装 → 模型推理/选择工具
→ 权限检查 → 工具执行 → 循环

上下文组装不是简单的"把当前文件发过去"，它包含两个被lodash memoize化的块：

- System Context：当前分支、默认分支、git用户名、git status

```
--short输出（截断到2000字符）、最近5条  
commit
```

```
- User Context: CLAUDE.md内存文件（四级  
层级结构）、当前日期
```

每次API调用前，这两个块都会被预先挂载，确保模型永远有全局视野。

2. 多模型分层架构：Haiku → Capybara → Opus

Claude Code内部模型调用，是三层模型逐级升级：

第一层：Haito（基于Haiku）

这是最小最快的模型，负责任务执行和计划检查。

当你开始一个编码任务时，首先由Haito接手，它快速执行，如果连续失败2次，就触发升级机制。

第二层：Capybara（内部代号）

一旦Haito搞不定，Capybara就上场了。

它是并行子任务分解器，能够把一个复杂任务，拆成多个子任务同时执行。

这些子任务之间共享prompt cache，意味着它们不需要各自重复传递上下文，context在多个子agent间是"热"的。

某种意义上，Capybara是真正的并行执行引擎。

第三层：Opus，最终决策者。

Opus只在Capybara也失败时才介入，它负责需要深度推理的复杂规划任务。

Haito失败2次 → 升级到Capybara →
Capybara失败 → 升级到Opus。

一句话总结：小事用小模型省成本，大事才动Opus。

还有一个官方没公开的opusplan：用`claude --mode opusplan`时，Opus负责做规划，Sonnet负责执行，既保证了规划质量，又降低了成本。

但这个组合不会出现在/model列表里，必须手动设置。

3. 管道生成系统

Claude Code不是简单调用grep、read、write这些单命令，

而是有一个专门的脚本构建逻辑在运行，动态生成工具管道。

比如当用户说：

"帮我找出日志里出现最多的错误"时，

模型不是调用一次grep完事，而是可能生成并执行一整条管道：

```
grep "error" app.log | awk '{print $1}' | sort  
| uniq -c | sort -rn
```

这条命令是模型根据当前任务动态决定的，不是预设模板。

模型觉得这样组合能达到目的，就生成并执行。

每个工具都有maxResultSizeChars参数限制返回大小。

当结果超过阈值时，内容自动写入临时文件，只返回一个文件路径给模型，而不是把全部内容塞进

context。这是防止context被撑爆的护栏。

这套设计的核心启发是：暴露可组合的管道能力，比暴露孤立的一次性命令API更有价值。

模型可以动态拼接工具，而不是每次都从零构造解决方案。

4. 5种Token压缩策略

当context window快满时，Claude Code不是简单地截断前文，而是有5种不同的压缩策略。

这意味着它的context管理是分层的：不同类型的内容，不同的压缩方式。

5. Subagent之间共享Prompt Cache

Capybara的设计中，并行子任务之间共享prompt cache。

这意味着：

- 任务A和任务B并行执行时，不需要各自传递完整的上下文
- Context是"热"的 (hot state)，在多个子agent间共享

- 子agent通过Task工具生成，有自己独立的循环，但可以访问父级缓存

源码中agents/和coordinator/目录，揭示了一个完整的多agent编排系统。

6. 工具权限系统

每个工具都有checkPermissions方法，执行前必须过审。

权限结果有三种：allow直接执行，ask弹确认框，deny直接拒绝。

Read-only工具（Read、Glob、Grep）在所有模式下自动通过。

真正强大的是钩子系统，PermissionRequest钩子，能返回updatedPermissions，直接重写整个权限系统的行为——不只是拦截，而是改写。

PostToolUse钩子更激进，能在模型看到工具输出之前，拦截并修改MCP响应。

本质上这是一套过滤器链，钩子不是被动拦截器，而是能主动改写流经数据的拦截器。

7. 对话存储与Compaction

对话转录为JSON存储在`~/claude/`，每个session有唯一ID，可通过`--resume <session-id>`恢复。

长对话会触发compaction，最旧的消息被summarization 压缩，但原始转录始终保留在磁盘上。

这意味着，你可以随时回到任何一个时间点的完整上下文。

8. 刻意为之的"缺陷"

Claude Code在启动时缓存了session日期，即使跨午夜也不会更新。

源码注释解释了原因：更新日期会让整个prompt cache失效。

宁可要旧数据，也不要cache invalidation。

这是性能优化中的经典trade-off——数据新鲜度 vs 系统稳定性。

Claude Code选择了稳定。

9. cc_workload hint

这其实是个挺有意思的发现。

Claude Code发给Anthropic API的每个请求里，都带了一个叫cc_workload的字段。

这个字段的值决定了，Anthropic的后端把你的请求放进哪个队列处理。

当你用cron定时触发Claude Code自动化任务时，这个字段会被标记为deferred——延迟队列，低优先级。

当你在终端里实时交互使用时，字段是正常优先级。

两个场景付费可能一样，但处理速度不一样。Anthropic在后台把cron job和自动化脚本这类"不紧急"的请求，降低了优先级。

说白了就是：

同样是调用Claude API，你手动交互的请求和跑脚本自动化的请求，享有的待遇不一样。

前者更快，后者更慢。

10. Structured Output重试循环

当你让Claude Code输出结构化数据时——比如JSON格式——模型有时候会输出格式错误的JSON，或者少了个逗号，或者括号不匹配。

Claude Code对这个场景有专门的容错设计：它会检测输出是否合法，如果不合法，自动重试，最多5次。

5次都失败才放弃。

这个次数可以自己改，比如改成10次：

```
MAX_STRUCTURED_OUTPUT_RETRIES=10
```

所以意思是：

Anthropic做这个工具的时候就知道，模型在输出严格格式时会有失败率，所以设计了自动重试机制，而不是直接报错丢给你。

12.MEMORY.md vs CLAUDE.md

两种记忆，两种哲学。

多数人知道CLAUDE.md是项目级系统提示，但源码揭示了一种更清醒的记忆哲学。

Claude Code的"记忆"靠的是显式持久化，而不是让模型自己记住。

源码里有句话很直接：

```
"Workers write to MEMORY.md files  
explicitly. The model never decides what  
to remember. Next session reads what you  
wrote. You choose what persists and what  
expires."
```

意思是：Agent不应该自己决定什么该记住。

你想让什么跨session存在，就写入MEMORY.md，下次Claude Code启动时主动去读。

你想让它忘掉什么，就不写或删除掉。

这是Karpathy提过的问题的答案：

Agent的内存不应该是隐式的，不能让模型自己决定什么重要。你是主人，模型是工人，记忆的写入权在你手里。

13.未发布的新功能

这是从源码里挖出来的还没发布的功能：

kairos：

一个还没发布的assistant mode。

能推送通知、能接GitHub webhooks、能在同一个项目里开多个并发session。

Undercover Mode：

为Anthropic自己员工准备的模式。

核心功能是隐藏代码归属，写进git的commit不会显示"Claude Code"或Anthropic的字样，写得像正常人类工程师的commit。（这。。。）

subscription-tiered parallelism：

分层并发订阅制。就是字面意思，你付不同级别的订阅费，能同时跑的并发任务数量不一样，贵的

plan能同时处理更多任务。

总结

这次泄露，不是模型泄露：没有模型权重，没有 system prompt 被暴露。

是工具链泄露：上层agent orchestration逻辑全部公开。

对于AI公司来说，这是最尴尬的那种泄露，竞争对手能看到你怎么做agent harness，但看不到你真正的护城河（模型本身）。

对于整个行业来说，这可能加速开源agent框架的进化（有人已经在基于这套源码构建自己的编程 agent了。）